

Day 8

Streams in C++

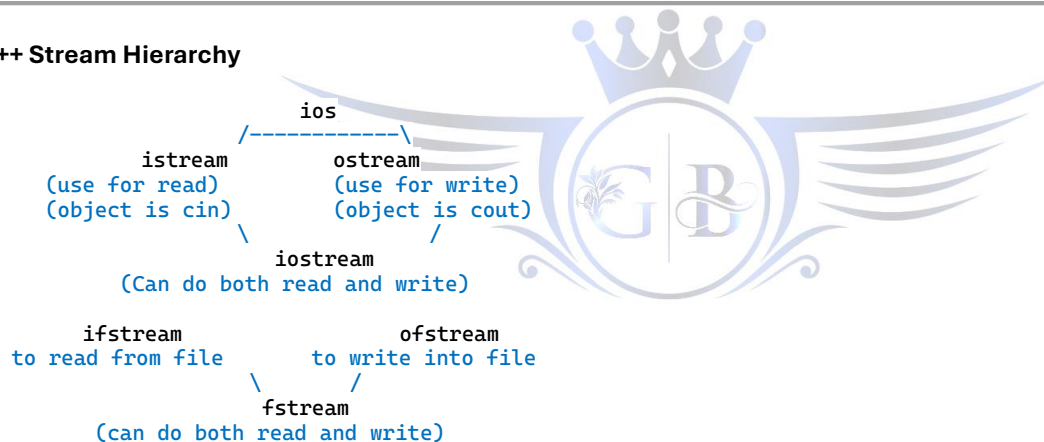
1. Purpose of File Handling

- File handling is required to **store data permanently** so that it can be used in the future.
 - In **C**, file handling is done using **library functions** like `fopen()`, `fclose()`, `fgetc()`, `fputc()`, etc.
 - In **C++**, file handling is implemented using **Object-Oriented Programming concepts** through **stream classes**.
-

2. Concept of Stream

- A **stream** is a **communication path** between a **source** and a **destination**.
 - When a program **reads** from a file, it opens an **input stream** on that file.
 - When a program **writes** to a file, it opens an **output stream** on that file.
-

3. C++ Stream Hierarchy



- `cin` is an object of **istream**.
 - `cout` is an object of **ostream**.
 - The hierarchy represents inheritance between stream classes.
-

4. Stream Classes in C++

Class Name	Purpose
ifstream	To read data from files (input stream).
ofstream	To write data to files (output stream).
fstream	To perform both read and write operations.

Header file:

```
#include <fstream>
```

5. Opening and Closing Files

Syntax:

```
ifstream in("filename");           //default mode ios::in
ofstream out("filename");           //Default mode ios::out | ios::trunc
fstream io("filename", ios::in | ios::out); //can do both read & write
```

Closing a file:

```
in.close();
out.close();
io.close();
```

6. Checking if a File Opened Successfully

C++ provides two ways to check if a file opened successfully:

1. Using overloaded operator !:

- `if (!in) { cout << "Cannot open file"; }`
- `if (!out) { cout << "Cannot open file"; }`

2. Using .fail() method:

- `if (in.fail()) { cout << "Cannot open file"; }`
- `if (out.fail()) { cout << "Cannot open file"; }`

3. Using .is_open() method:

- `if (file.is_open()) { cout << "File exists."; }`
 - `else { cout << "File does not exist."; }`
-

7. Basic File I/O Functions

Character-Level I/O

- **Write a single character:**

- `ostream& put(char ch);`

- **Read a single character:**

- `istream& get(char& ch);`

Line-Level I/O

- **Read a line:**

- `istream& getline(char* buffer, streamsize n);`

- **Write a line:**

- `out << "string" << endl;`
-

8. Binary File I/O

Binary Write

```
ostream& write(char* buffer, streamsize n);
```

Binary Read

```
istream& read(char* buffer, streamsize n);
```

Example Syntax:

```
out.write(reinterpret_cast<char*>(&object), sizeof(object));  
in.read(reinterpret_cast<char*>(&object), sizeof(object));
```

To count number of bytes read:

```
i.gcount(); // returns bytes read in last read operation
```

9. Serialization and Deserialization

- **Serialization:** The process of **saving (persisting)** an object's data into a file.

```
f.write((char*)&obj, sizeof(obj));
```

- **Deserialization:** The process of **retrieving** the object's data from the file.

```
f.read((char*)&obj, sizeof(obj));
```

10. File Opening Modes

Mode	Meaning
ios::in	Opens file for reading
ios::out	Opens file for writing
ios::app	Opens file for appending (data always added to end)
ios::ate	Opens file and seeks to end initially (allows read/write anywhere)
ios::trunc	Destroys existing file and creates a new one if it does not exist
ios::binary	Opens file in binary mode (no text translation)

Default modes:

```
ifstream → ios::in  
ofstream → ios::out | ios::trunc  
fstream → ios::in | ios::out
```

11. Difference Between ios::out and ios::trunc

- **ios::out:** Opens the file for writing. If the file exists, contents may be truncated (depends on implementation).
- **ios::trunc:** Always **clears** (truncates) the file when opened.
- Use:
 - **ios::out | ios::trunc** // ensures overwriting file every time

12. Difference Between `ios::app` and `ios::ate`

Mode	Description
<code>ios::app</code>	Opens the file for appending only . Data is always written at the end, regardless of file pointer movement.
<code>ios::ate</code>	Opens the file and positions the file pointer at the end but allows read/write operations anywhere in the file after seeking.

13. Random Access in Files

Each file stream has two internal pointers:

- **Get Pointer (g)** → Indicates the next position for **input (read)** operations.
- **Put Pointer (p)** → Indicates the next position for **output (write)** operations.

Pointer Manipulation Functions

Function	Description
<code>seekg(offset, direction)</code>	Move get pointer
<code>seekp(offset, direction)</code>	Move put pointer
<code>tellg()</code>	Return current position of get pointer
<code>tellp()</code>	Return current position of put pointer

Direction options:

- `ios::beg` → from beginning of file
- `ios::cur` → from current position
- `ios::end` → from end of file

Example:

```
fstream_obj.seekg(0, ios::beg);  
fstream_obj.seekp(0, ios::end);
```

14. Stream State and Re-Reading Files

When the file is read until the end, the **eofbit** (End-of-File flag) is set internally. Once set, no further reading is possible until it's cleared.

To clear the flag:

```
ff.clear();           // clears all error flags (e.g., eofbit, failbit)  
ff.seekg(0, ios::beg); // reposition the get pointer
```

This allows reading the file again from the beginning.

15. Text Mode vs Binary Mode

(a) Newline Handling

Text Mode:

- While writing, newline (\n) is converted to \r\n (carriage return + line feed).
- While reading, \r\n is converted back to Nnewline (\n).

Binary Mode:

- No conversion occurs.

(b) End-of-File Representation

Text Mode:

- End of file represented by special character with value 26 (Ctrl+Z).

Binary Mode:

- No such representation exists.

(c) Numeric Data Storage

Text Mode:

- Numeric values are stored as individual characters.
- Example: 1234.567 → 8 bytes (each digit + decimal point considered as character).

Binary Mode:

- Numeric values are stored in the computer's internal (memory) representation.
- Example: float 1234.567 → 4 bytes.

(d) Compatibility

- Text and binary modes are **not compatible** with each other.

16. Important I/O Operators and Functions

Operation	Function / Operator
Write data to file	ofstream::put(char) / ostream::write() / <<
Read data from file	ifstream::get(char&) / istream::read() / >>
Write entire structure	write((char*)&object, sizeof(object))
Read entire structure	read((char*)&object, sizeof(object))
Check number of bytes read	gcount()
Clear flags	clear()
Seek in file	seekg() / seekp()
Get pointer position	tellg() / tellp()

Close file	close()
------------	---------

17. Operator Overloading for File I/O (User-defined Types)

For user-defined classes, overload insertion and extraction operators to enable direct file or console I/O:

```
friend ostream& operator<<(ostream&, ClassName&);
friend istream& operator>>(istream&, ClassName&);
```

18. Summary Table

Aspect	Description
Stream Base Class	ios
Input Classes	istream, ifstream
Output Classes	ostream, ofstream
Input/Output Combined	iostream, fstream
Default Modes	ifstream → in, ofstream → out
Binary Mode	Prevents newline and EOF conversions
app vs ate	app → always append; ate → open at end but can move pointer
out vs trunc	trunc forces deletion of existing file
clear()	resets error state (eof, fail, bad bits)
Serialization	writing object to file
Deserialization	reading object from file
seekg/seekp	move get/put pointer
tellg/tellp	get pointer position
Text vs Binary	newline, EOF, data size handling differ

Manipulators in C++

Manipulators are used in C++ to **control the formatting of input/output streams**.

They are defined in the header `<iomanip>` (for advanced manipulators) and `<iostream>` (for basic manipulators).

1. Number System (Base Format) Manipulators

Manipulator	Description	Syntax Example	Output / Effect
dec	Displays value in decimal (base 10)	<code>cout << dec << 100;</code>	100
oct	Displays value in octal (base 8)	<code>cout << oct << 100;</code>	144
hex	Displays value in hexadecimal (base 16)	<code>cout << hex << 100;</code>	64
setbase(n)	Displays value in specified base (8, 10, or 16)	<code>cout << setbase(16) << 100;</code>	64

Note:

`setbase(10)` → Decimal, `setbase(8)` → Octal, `setbase(16)` → Hexadecimal.

2. Field Width and Output Alignment Manipulators

Manipulator / Function	Description	Syntax Example	Output / Effect
setw(n)	Sets field width for next output	<code>cout << setw(10) << 123;</code>	123
width(n)	Member function version of <code>setw(n)</code>	<code>cout.width(10); cout << 123;</code>	123
fill(ch)	Fills empty field spaces with given character	<code>cout.fill('*'); cout << setw(10) << 123;</code>	*****123
setfill(ch)	Manipulator version of <code>fill()</code>	<code>cout << setfill('*') << setw(10) << 123;</code>	*****123
ios::left	Left aligns output	<code>cout.setf(ios::left); cout << setw(10) << 123;</code>	123
ios::right	Right aligns output (default)	<code>cout.setf(ios::right); cout << setw(10) << 123;</code>	123
ios::internal	Sign left-aligned, number right-aligned	<code>cout.setf(ios::internal); cout << setw(10) << -123;</code>	- 123

Notes:

- `setw()` / `width()` affect **only the next output operation**.
- `fill()` remains **in effect until changed**.
- Alignment is managed using **format flags** (`ios::left`, `ios::right`, `ios::internal`).

3. Floating-Point Manipulators

Manipulator	Description	Syntax Example	Output / Effect
setprecision(n)	Sets total number of digits (default) or decimal digits (with fixed)	cout << setprecision(3) << 123.456;	123
fixed	Uses fixed-point notation; precision → digits after decimal	cout << fixed << setprecision(3) << 123.456;	123.456
scientific	Displays floating values in scientific (exponential) form	cout << scientific << 1234.56;	1.234560e+03
showpoint	Forces display of decimal point and trailing zeros	cout << showpoint << 25.0;	25.0000
noshowpoint	Suppresses decimal point if not required (default)	cout << noshowpoint << 25.0;	25
showpos	Forces display of '+' sign for positive numbers	cout << showpos << 45;	+45
noshowpos	Hides '+' sign (default behavior)	cout << noshowpos << 45;	45
uppercase	Displays 'E' and hex digits in uppercase	cout << uppercase << hex << 255;	FF
nouppercase	Displays 'e' and hex digits in lowercase (default)	cout << nouppercase << hex << 255;	ff

Notes:

- setprecision(n) controls **significant digits** unless fixed or scientific is applied.
- fixed and scientific remain active until explicitly changed.

4. Stream Control and Miscellaneous Manipulators

Manipulator	Description	Syntax Example	Output / Effect
endl	Inserts a newline and flushes output buffer	cout << "Hello" << endl;	Hello (new line)
ends	Inserts null character ('\0') at end of output	cout << "Hi" << ends;	Hi\0
flush	Flushes output buffer without newline	cout << "Hello" << flush;	Ensures immediate display
boolalpha	Displays boolean values as true or false	cout << boolalpha << true;	true
noboolalpha	Displays boolean values as 1 or 0	cout << noboolalpha << true;	1
setiosflags(flag)	Sets multiple formatting flags simultaneously	` cout << setiosflags(ios::fixed	ios::showpoint); `

resetiosflags(flag)	Clears specified format flags	cout << resetiosflags(ios::fixed);	Restores default format
----------------------------	-------------------------------	---------------------------------------	-------------------------

5. Common Combinations (For Practical Use)

Formatting Goal	Manipulator Combination	Example Syntax
Display in hexadecimal	hex or setbase(16)	cout << hex << 255;
Display with 3 decimal places	fixed, setprecision(3)	cout << fixed << setprecision(3) << value;
Display table-like output	setw(), setfill(), ios::left	cout << left << setw(10) << setfill(' ') << name;
Show positive signs	showpos	cout << showpos << num;
Left-align names, right-align marks	setw(), ios::left, ios::right	cout << left << setw(10) << name << right << setw(5) << marks;
Scientific notation with uppercase	scientific, uppercase	cout << scientific << uppercase << 123.45;
Boolean output as text	boolalpha	cout << boolalpha << true;

6. Important Behavioral Notes

Aspect	Explanation
Persistence	setw() affects only the next insertion; fill(), precision(), and format flags persist until changed.
Headers Required	<iostream> for basic manipulators (endl, flush, etc.); <iomanip> for advanced manipulators (setw, setprecision, etc.).
Chaining	Multiple manipulators can be chained together: cout << fixed << setprecision(2) << setw(10) << setfill('*') << value;
Output Flushing	endl and flush both clear the output buffer; endl adds a newline, flush does not.

7. Example Summary of Visual Effects

Manipulator	Before	After
setw(8)	123	123
setfill('*')	123	*****123
ios::left	123	123
setprecision(2) + fixed	1.2345	1.23

hex	255	ff
uppercase + hex	255	FF
showpos	45	+45
boolalpha	true	true
endl	Hello	Hello (new line)

8. Quick Reference: Commonly Used Manipulators

Category	Manipulators
Base Format	dec, oct, hex, setbase()
Width & Alignment	setw(), width(), fill(), setfill(), ios::left, ios::right
Precision & Format	setprecision(), fixed, scientific, showpoint, showpos
Stream Control	endl, flush, ends, boolalpha, setiosflags(), resetiosflags()

